

AST 4762/5765: Astronomical Data Analysis

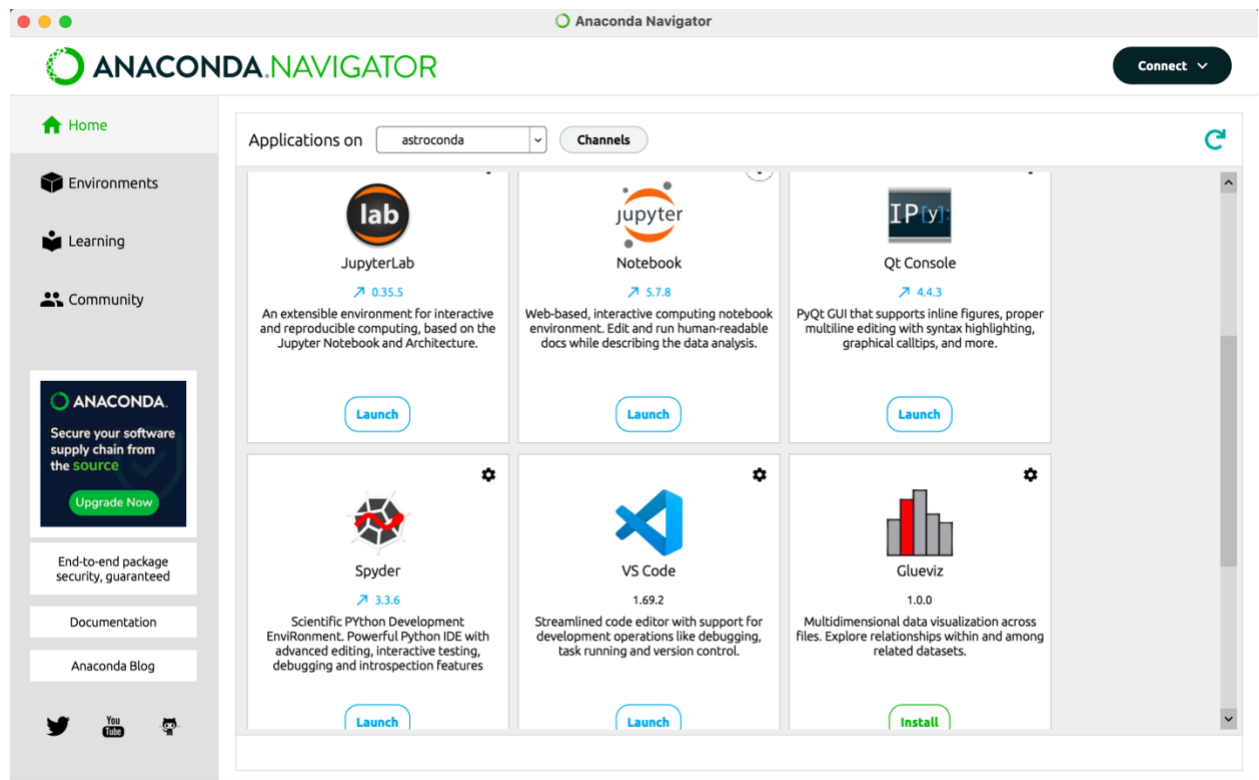
Programs we will use in class & installation.

In this class we will use Python and revision control. For this reason, you will need to get access to a [Python](#) distribution and [GitHub](#).

Python:

1. If you have enough disk space, download the full Anaconda distribution from here: <https://www.anaconda.com/products/distribution>. This comes with all the packages we will use in the class, and some additional packages you may use in the future. Make sure to get the 64-bit version.

Once the installation is finished, use your Windows start icon (for Windows users) or your Spotlight Search or the Applications (for Mac users) for the Anaconda Navigator. If you are a Unix user, see the Ubuntu folder for further instructions. Once you launch the Navigator you will get something that looks like this:



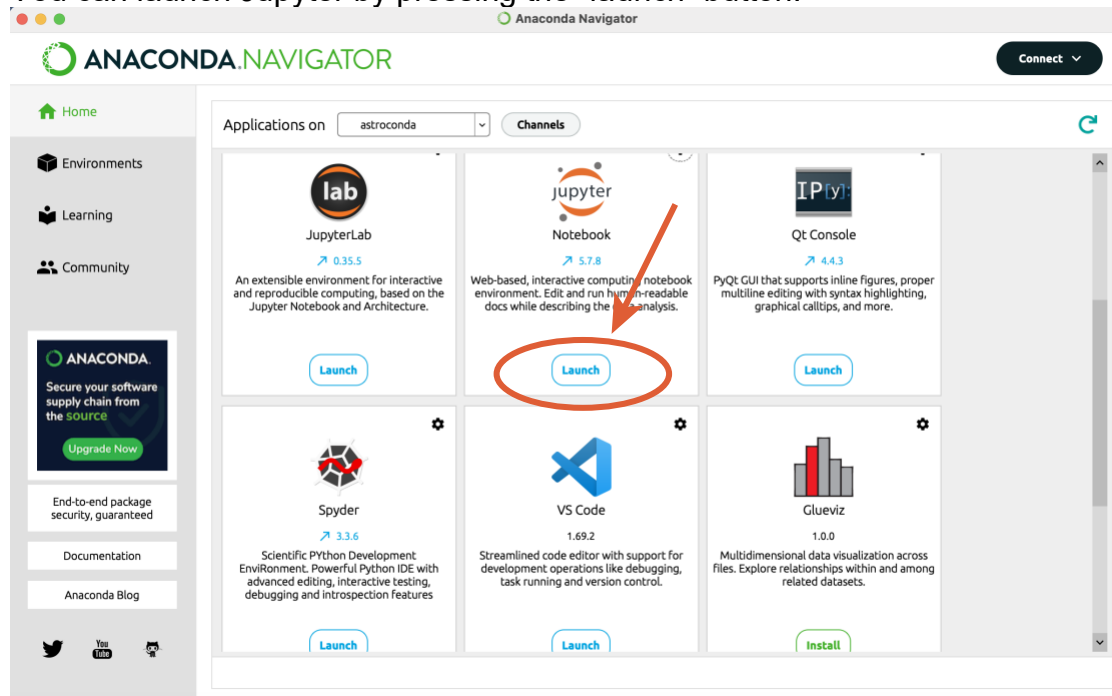
All the programs that are installed in your computer will have the "Launch" button underneath. Programs that you can install in the future have an "Install" button underneath.

In this class you will deliver your homework as a .py file, which will need to run without issues from the command line and print out all requested results (we will discuss all details in class).

However, to write, run and debug your code there are a few options you can chose from. If you prefer an interactive GUI you can use: Jupyter notebooks (directly from Anaconda), Jupyter notebooks in VSCode and Spyder. If you prefer a more bare-bones, Unix experience you can use emacs or another text editor and the command line.

a) Jupyter notebooks:

You can launch Jupyter by pressing the “launch” button:

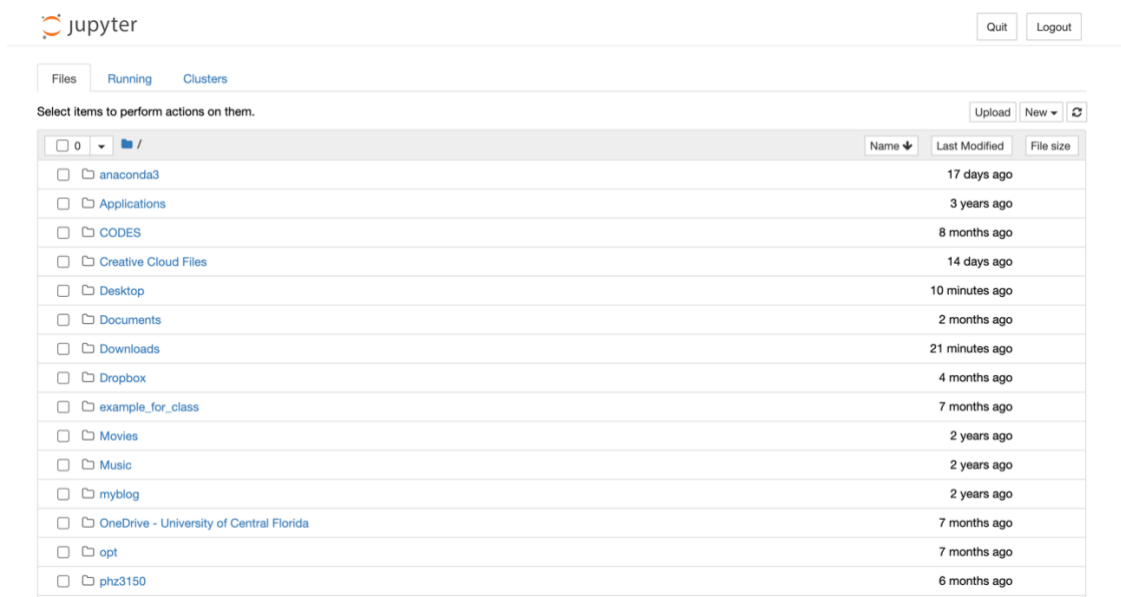


or you can use the Anaconda prompt (Windows) or a Terminal (Mac) and type:
`jupyter notebook`

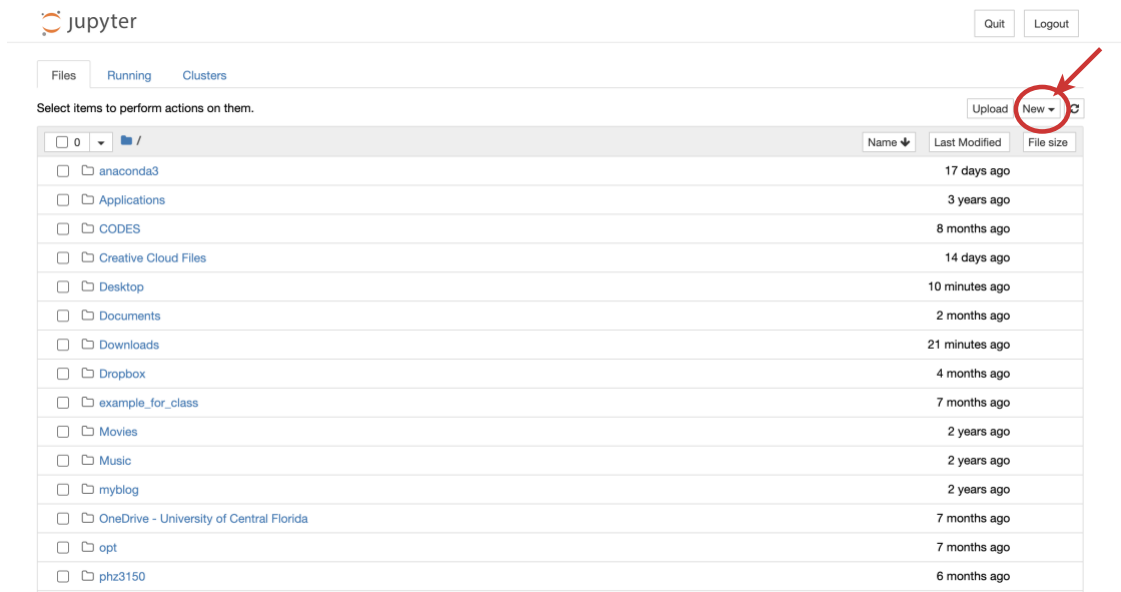
```
theodora_karalidi — -bash — 123x32
~/CODES/TERRESTRIAL/PROGRAM/GAP — -bash ...
Last login: Thu Aug 11 13:24:15 on ttys010
core file size      (blocks, -c) 0
data seg size       (kbytes, -d) unlimited
file size           (blocks, -f) unlimited
max locked memory   (kbytes, -l) unlimited
max memory size     (kbytes, -m) unlimited
open files          (-n) 2560
pipe size           (512 bytes, -p) 1
stack size          (kbytes, -s) 8192
cpu time            (seconds, -t) unlimited
max user processes  (-u) 2784
virtual memory      (kbytes, -v) unlimited

The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
(base) Theodoras-MacBook-Pro:~ theodora_karalidi$
(base) Theodoras-MacBook-Pro:~ theodora_karalidi$
(base) Theodoras-MacBook-Pro:~ theodora_karalidi$ jupyter notebook
```

This will start a new tab in your default web browser. It will look like this:



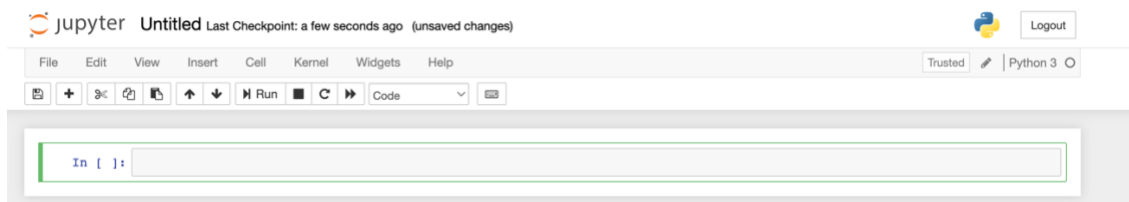
Use this window to navigate to the folder you want to work in (e.g., the ast4762/homework/ folder). Once there, go to the top right corner and press the “New” button:



And then the Python 3 button:

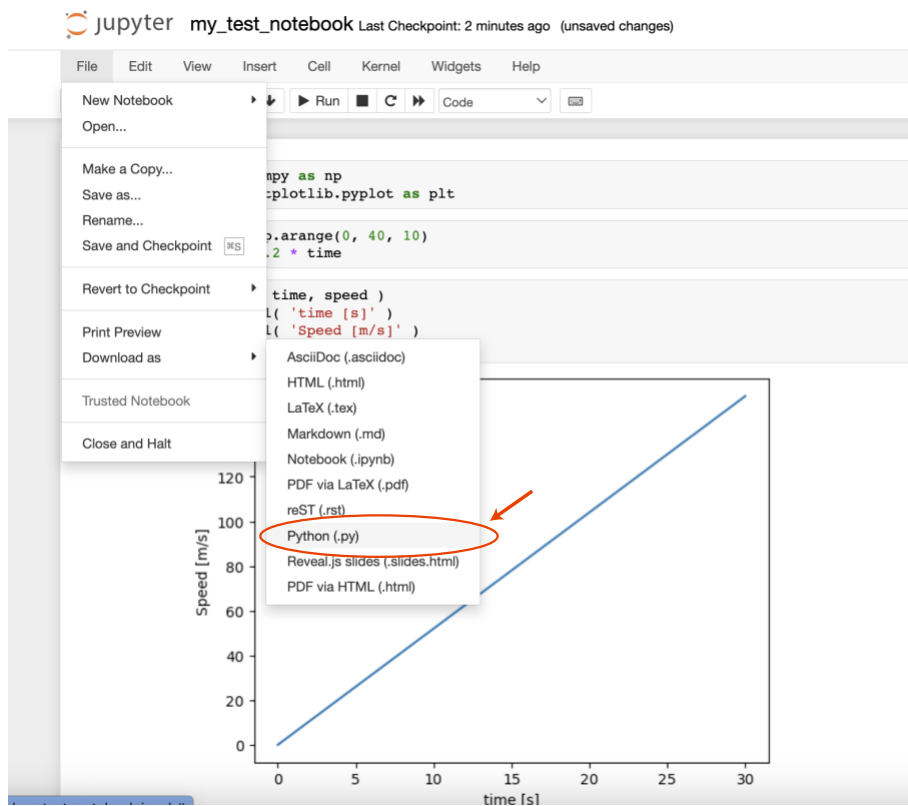


This will launch another tab with your Jupyter notebook:



We will recap how to change the name of the notebook, code and run programs in it in the classroom.

When your homework is ready for submission (code written and debugged), you will need to save the file as a .py file. To do this go to File → Download as → .py:



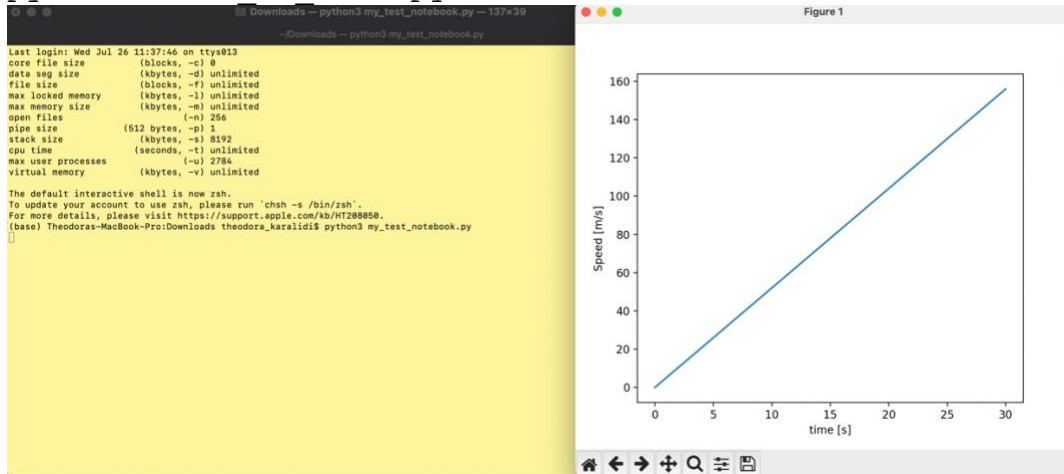
This will save your code as a plain text .py file that will look like this:

```
my_test_notebook.py
1  #!/usr/bin/env python
2  # coding: utf-8
3
4  # In[1]:
5
6
7  import numpy as np
8  import matplotlib.pyplot as plt
9
10
11 # In[2]:
12
13
14 time = np.arange(0, 40, 10)
15 speed = 5.2 * time
16
17
18 # In[3]:
19
20
21 plt.plot( time, speed )
22 plt.xlabel( 'time [s]' )
23 plt.ylabel( 'Speed [m/s]' )
24 plt.show( )
25
26
```

header letting the system know how to interpret the following lines (i.e., as a Python code)

To run the code now, I can go my Unix command line and type:

`python3 name of file.py:`

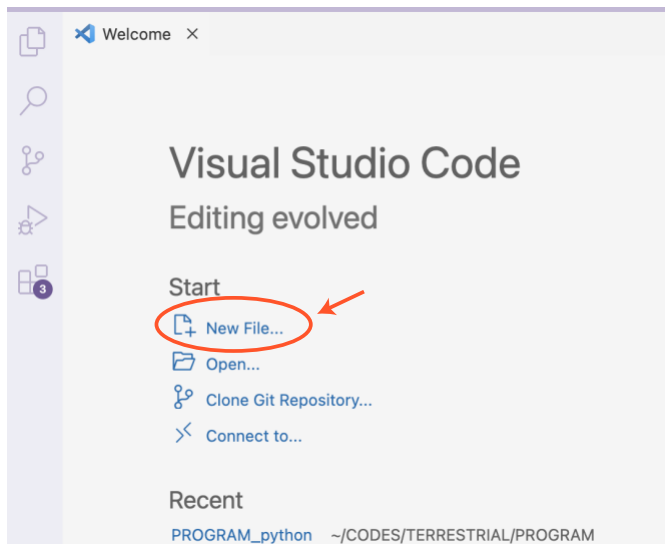


Note that the code runs, and a window opens to show my plot.

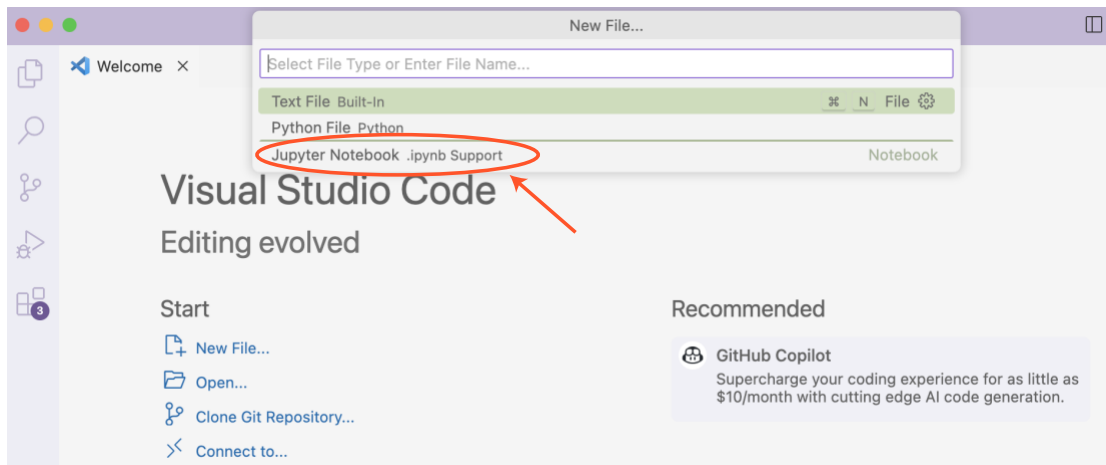
b) VS Code:

Visual Studio Code allows you to run your code as a plain text, notebook, etc. Here we show how to create a Jupyter notebook.

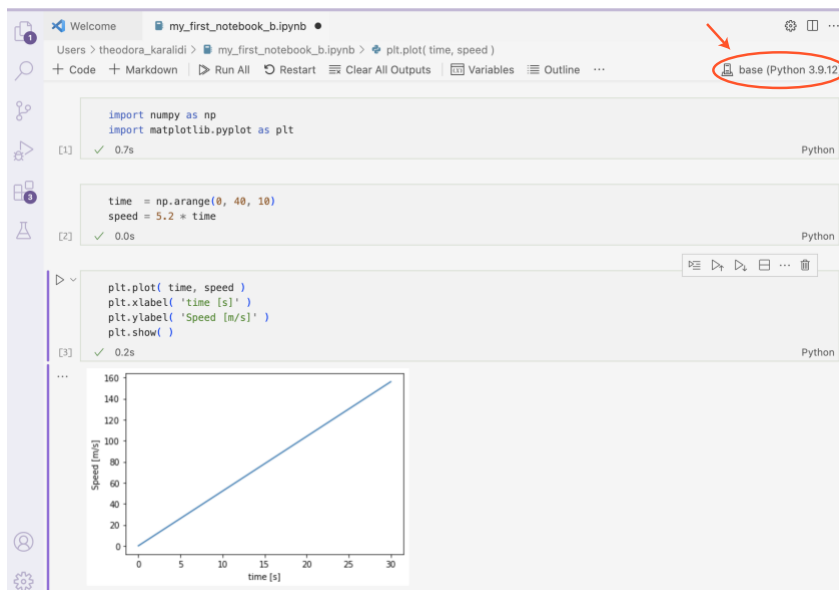
Launch VSC and go to New File:



then select the Jupyter notebook:



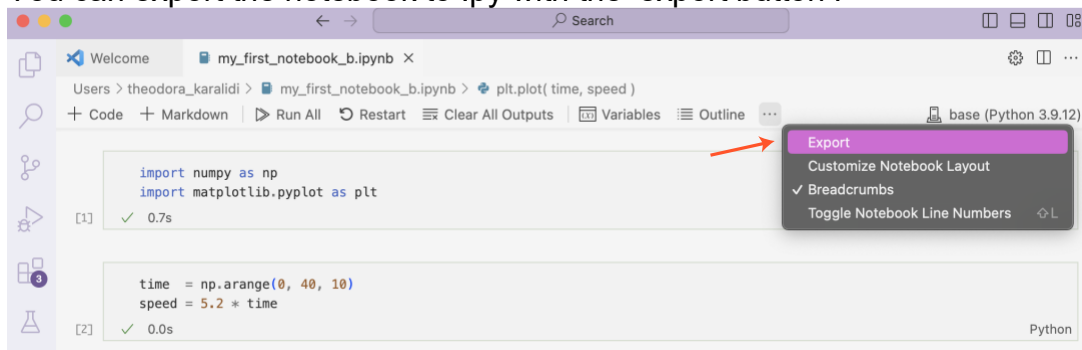
this will launch a Jupyter notebook like this one (note I have already filled it in with the code here):



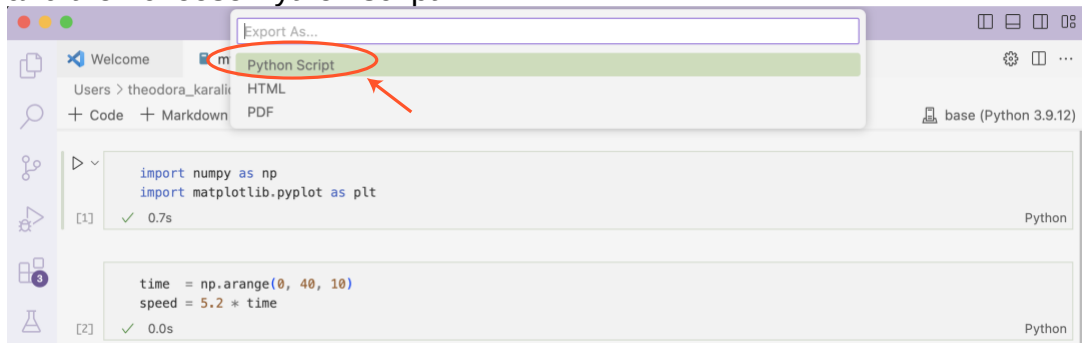
[Also, note that you might need to choose on the top right corner the python kernel you want to use (Python: select interpreter → chose the Python 3 installation you did)]. Then you write and run the code like you do with the plain Jupyter notebook.

A few benefits of VSC are that you can integrate Git, it shows your directly the run time of your code and it has integrated debugging tools. Additionally, if you have a Windows laptop with WSL installed (see [Windows_WSL_instructions.pdf](#)) you can use VSC to work directly on your Unix section (see: <https://code.visualstudio.com/docs/remote/wsl>).

You can export the notebook to .py with the 'export button':



and then choose Python script:



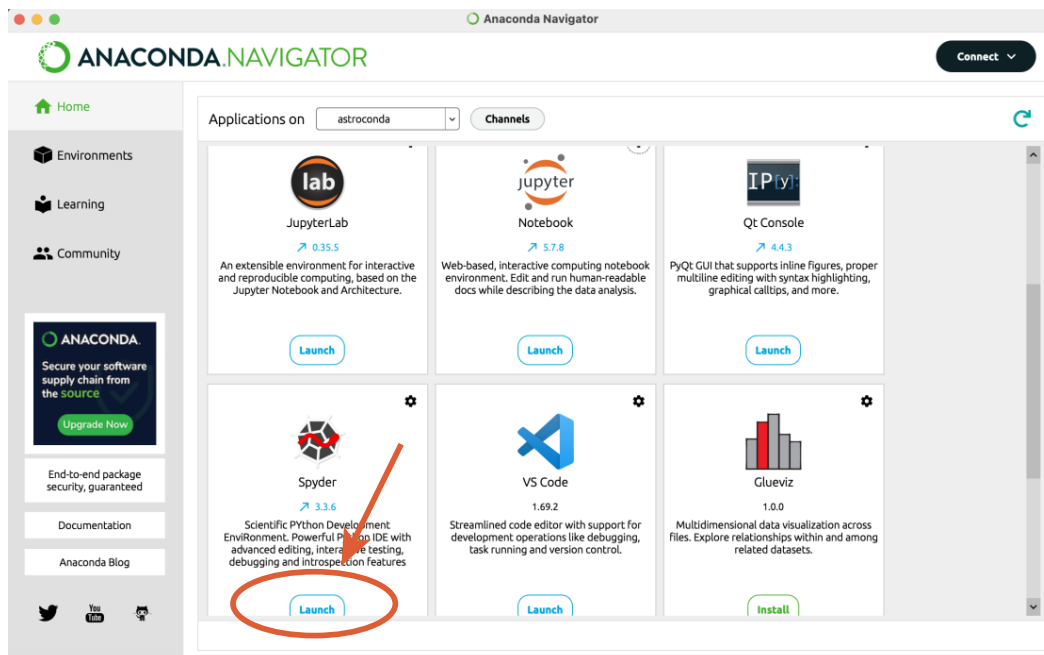
Save the file (e.g., my_first_notebook_b.py). Then you can use again:

```
python my_first_notebook_b.py
```

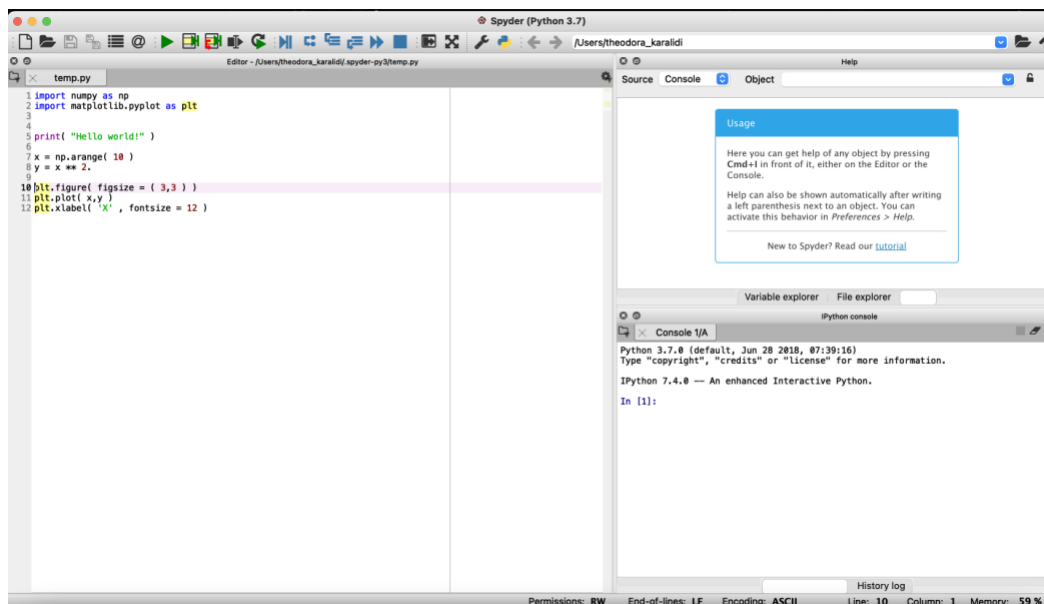
on the terminal and see your code run.

c) Spyder:

If you prefer, you can code your homework using Spyder. Spyder offers the ability to use a text editor to write your code and a terminal to run the code in one application. To start Spyder, just click on the "Launch" button under Spyder:



This will launch Spyder. It will look like this:



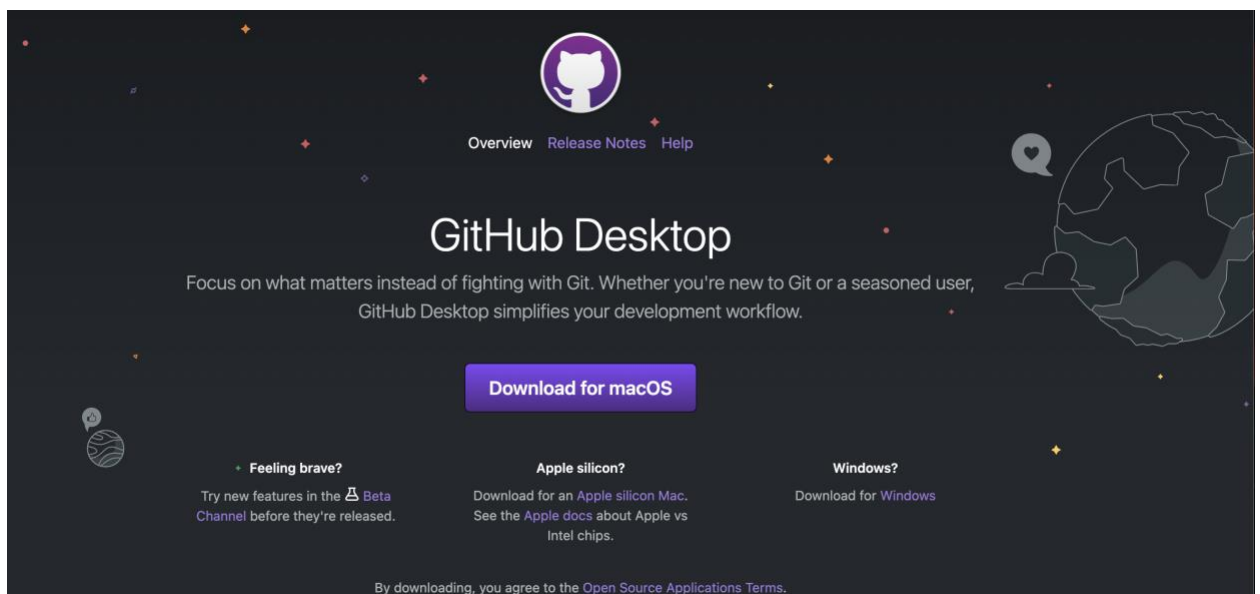
We will talk in class about how to use this to code.

2. Note that if you prefer a lighter installation, you can install Miniconda following the steps of section 2 in the *win10-software.pdf* or *mac-software.pdf*. The

installation process differs slightly (e.g., you will need to install numpy, matplotlib and scipy separately. See the pdfs), and you can only start the Jupyter notebook using the Prompt or Terminal.

GitHub:

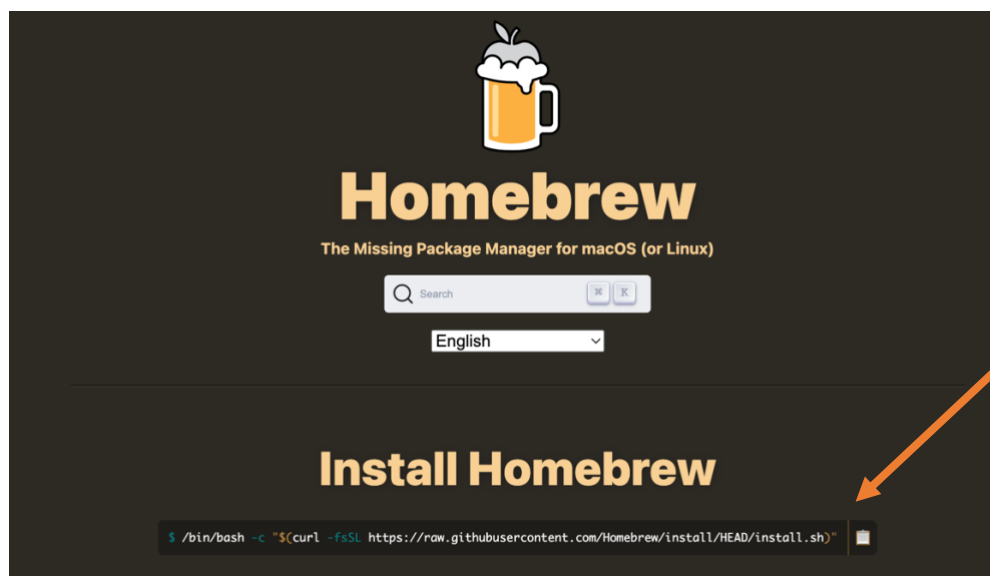
In this class we will use GitHub for revision control. You will need to install the GitHub Desktop app from here: <https://desktop.github.com/> . We will discuss in class how to use GitHub. In the meantime, if you prefer to experiment on your own and make an account with GitHub, **please make the account using your @ucf email**. This will be important for the GitHub classroom we will use later on and allows you to get a free pro account!



Unix:

Getting familiar with the Unix shell is important if you do research. While GUIs are great, you cannot use them to communicate with computer clusters like our own Stokes and Newton. If you will be doing computational heavy work clusters are crucial. In this class we will use the Unix shell to test that the finalized code runs without issues (this is also how your code will be graded!) and zip/tar the homework folder you hand in Webcourses.

1. If you work with a Unix or Mac (it is Unix under the hood) laptop you already have a terminal set up. You can search for the “Terminal” if you don’t have it already on your dock.
 - a. If you have a Unix laptop you are already done.
 - b. If you have a Mac you will need to get access to the “Command line tools”. The easiest way to do that is via Homebrew. Homebrew also gives you access to a multitude of other handy packages. To install Homebrew go to: <http://brew.sh/>
At the top of the page, you will see a line of shell script to paste into a terminal:



You may need to supply a password. If you haven't already installed the Command Line Tools, you'll see a message that "The XCode Command Line Tools will be installed." Press Enter to continue when prompted by the Homebrew installation script. You now have access to the command line and hundreds of open-source packages available!

2. If you work on a Windows 11 laptop you will need to follow the `windows_wsl_instructions.pdf` to get access to Unix.